Query Processing

1 Introduction

Definition: *Query processing* refers to the range of activities involved in extracting data from a database efficiently and correctly.

These activities include:

- Translating queries written in high-level languages (like SQL) into low-level internal representations.
- Applying optimization techniques to select efficient query execution plans.
- Evaluating the optimized plan to produce the final result.

2 Overview of Query Processing

The basic steps involved in query processing are illustrated in Figure 1. There are three major phases:

1. **Parsing and Translation**
2. **Optimization**
3. **Evaluation**

Before execution, the query must be transformed into an internal format. Although SQL is user-friendly, it is not suitable for the system's internal representation. Hence, databases typically convert it into an equivalent expression based on the **Extended Relational Algebra**.

3 Step 1: Parsing and Translation

This phase converts the user's SQL query into an internal representation. It functions similarly to the parsing stage in a compiler.

Tasks Performed by the Parser

- Checks the **syntax** of the user's query.
- Verifies that relation and attribute names exist in the database schema.
- Builds a **parse tree** representation of the query.

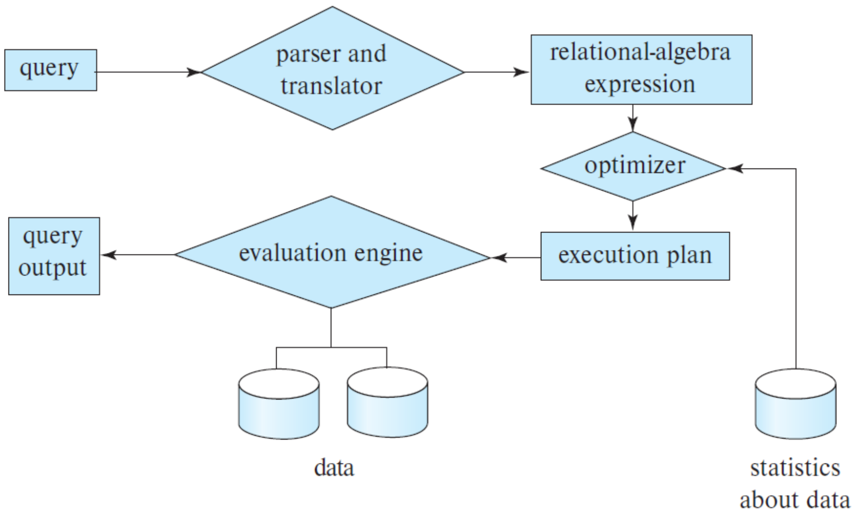

Figure 1: Major Steps in Query Processing

- Translates the parse tree into an equivalent **relational-algebra expression.**

If the query involves views:

- For normal views, the system replaces the view reference with the relational-algebra expression defining it.
- For **materialized views**, the stored results are directly used.
- **Recursive views** are processed via a fixed-point computation method.

4 Step 2: Query Optimization

Given a query, there can be multiple methods to produce the same result. Optimization identifies the **most efficient query plan** among all alternatives.

Example Query

```
SELECT salary FROM instructor WHERE salary < 75000;
```

This SQL query can be represented using two equivalent relational-algebra expressions:

$$\sigma_{salary<75000}(\Pi_{salary}(instructor))$$

$$\Pi_{salary}(\sigma_{salary<75000}(instructor))$$

Execution Choices

Each operation can be implemented using various algorithms:

- Performing a **full table scan** to check every tuple.
- Using a **B^+-tree index** on `salary` to directly locate relevant tuples.

Key Terms:

- **Evaluation Primitive:** A relational-algebra operation annotated with specific implementation details (algorithm, index, etc.).
- **Query-Evaluation Plan (or Query-Execution Plan):** A sequence of evaluation primitives defining the complete execution strategy for a query.

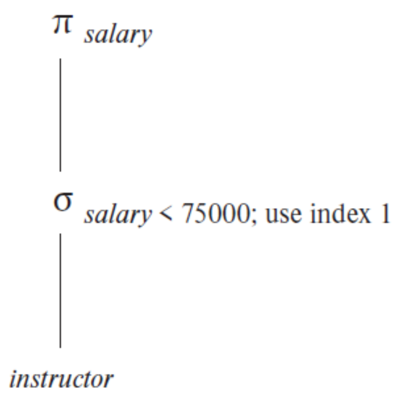

Figure 2: Example of a Query-Evaluation Plan using an Index

In Figure 2, the plan specifies that the selection operation should use *index 1* on `salary` for faster access.

5 Step 3: Query Evaluation

Once a plan is selected, the **Query Execution Engine** executes it and returns the result to the user.

Different plans may vary greatly in cost, so the database system's goal is to minimize the overall resource usage. This selection process is called **Query Optimization**.

Even though internal representations differ (relational algebra or annotated parse trees), the underlying principles of parsing, optimization, and evaluation are common to all modern DBMSs.

6 Cost Estimation and Optimization

The optimizer must estimate the **cost** of alternative query plans. While exact costs depend on runtime factors (e.g., available memory, disk I/O, caching), databases use approximate cost models based on:

- Number of disk accesses.
- CPU computation time.
- Network communication overhead (in distributed databases).

Accurate cost estimation enables the optimizer to choose the plan with the minimum estimated cost.

Summary

- Query processing involves **parsing**, **optimization**, and **evaluation**.
- Queries are internally represented using **extended relational algebra**.
- **Query Optimization** identifies the most efficient execution plan.
- **Evaluation Plans** consist of annotated primitives detailing execution strategies.
- Cost estimation plays a crucial role in selecting optimal plans.